

CREATION D'UNE APPLICATION JAVA :
APPLICATION DE GESTION DE FACTURES
D'ELECTRICITE : EDI ENERGIE

DOCUMENT TECHNIQUE



edi energie

L'ENERGIE QUI VOUS ECLAIRE

EDI ENERGIE

PARTICIPANTS :

- LEUDJEU Eloisa
- TAKAM Duvarelle
- CHERTY Yassine

CLASSE : CPI1-
GROUPE 3IL

ANNEE : 14/3/2026-
29/05/2026

Sous la supervision
de :

Mr Beverini

SOMMAIRE

INTRODUCTION

- I. ENVIRONNEMENT TECHNIQUE
- II. ARCHITECTURE GENERALE
- III. FLUX PRINCIPAL DE L'APPLICATION
- IV. BASE DE DONNEES

INTRODUCTION

Ce document présente l'architecture technique et le fonctionnement interne de l'application de gestion de facture < **EDI ENERGIE** >. Cette application permet à un utilisateur de garder ses factures et renvoyer le montant à payer de sa consommation (en kilowatt) chaque mois.

I. ENVIRONNEMENT TECHNIQUE

Cette application développée en langage Java 21 à l'aide de l'IDE appelé Eclipse dans un environnement JDK-21, avec une interface graphique basée sur la bibliothèque Swing intégré dans cet environnement, et une base de données SQLite pour la persistance des données créée par le biais du driver JDBC sqlite-jdbc. Elle est uniquement téléchargeable sur un environnement JDK-21. Elle utilise des bibliothèques spécifiques à savoir iTextPDF pour la génération du PDF et java.awt.print pour l'impression.

II. ARCHITECTURE GENERALE

Cette application repose sur un code composé de 08 classes principales à savoir :

1. La classe CLIENT

Cette classe représente les données entrées par un client dans l'application. Elle affiche les informations du client (son nom, son Id, son adresse) à travers la méthode **afficher**. Elle stocke aussi ses informations dans une base de données créée à travers la méthode **sauvegarder**

2. La classe COMPTEUR

Cette classe représente les données d'un compteur électrique associé à ce client. Elle permet à l'utilisateur d'entrer son numéro du compteur qui devra être uniquement de 14 chiffres (la taille normale du numéro de compteur) une vérification qu'on a eu à mettre dans le setter de la propriété associé au numéro du compteur, sa consommation (en KWh) qui est un nombre positif vérifié dans un setter associé. Elle récupère aussi toutes ces valeurs grâce aux getters tout en les stockant dans la base de données à partir de la requête SQL associée par La méthode `sauvegarder`.

3. La classe FACTURE

La classe Facture crée une facture électrique. Elle calcule son coût selon la consommation du client (qui est le reste trouvé entre la première consommation et la deuxième consommation sachant que la première consommation d'un nouvel client est 0) entre en KWh sachant que le kilowatt est placé à 15 centimes (prix en euro) à travers la méthode `calculermontant`, et affiche les détails de la facture à l'écran par la méthode `afficher`. Elle génère la facture en PDF à travers la méthode `generer` et sauvegarde toutes les factures dans la base de données grâce à une requêtes SQL.

4. La classe ConnexionDB

Cette classe sert à connecter l'application à la base de données. Elle est utilisée par toutes les autres classes.

5. La classe ConnexionGU

Cette classe correspond à l'interface de connexion de l'application. Elle crée une fenêtre divisée en deux sections dont la première section est celle de la `connexion` créée par la méthode `buildPanelConnexion` et comportant la création des champs (ID de connexion et mot de passe) et un bouton «`se connecter` » créé par la méthode `builonglets` pour donner l'accès à l'interface

principale. Et la deuxième portant le nom de **première connexion** créée par **buildPanelInscription** qui donne le droit à l'utilisateur de créer un compte et elle comporte la création des champs (nom, Id, adresse et mot de passe) et un bouton << **créer un compte**>> crée par la méthode **builonglets** également. Cette classe prend en compte la vérification de l'ID du client et de son mot de passe enregistrés dans la base de données lorsque l'utilisateur essaye de se connecter à son compte par la méthode **tenterconnexion**. Elle vérifie aussi le dernier ID utiliser pour le remplir automatiquement à la prochaine ouverture.

6. La classe Id

Cette classe génère automatiquement des nombres auto-incrémentés considérés comme des identifiants à travers la méthode **getandIncrement**.
Exemple : Le premier utilisateur inscrit reçoit l'ID 1000, le second reçoit l'ID 1001 et ainsi de suite.

7. La classe InitDB

Elle a créé la base de donnée de l'application qui comporte 4 tables à savoir : clients, compteurs, factures, utilisateurs.

8. La classe MainGU

La classe MainGU est la classe principale de notre projet car elle crée l'interface principale de notre application. Elle ouvre une fenêtre divisée en 4 sections soit :

- La première section comporte le titre de l'application et du logo ainsi que le bouton <<imprimer>> et << exporter PDF>> tout ceci crée dans le constructeur de la classe et la méthode **buildCenter**. Les fichiers sont importés en PDF par la méthode **exporterpdf** et imprimer par la méthode **imprimerRecu**.
- La deuxième section gère l'affichage des informations du client connecté à travers la méthode **buildficheclient** e, elle crée une nouvelle facture grâce au bouton **+Nouvelle facture** générer la classe **border** dans la

méthode `buildsidebar` et quitte l'application par le bouton << Quitter>> créée par

- La troisième section gère l'aperçu du reçu par les méthodes `recuvide` (aide à la sélection de la facture), `afficherRecu` (affiche le contenu de la facture), et `buildrecupanel` (qui s'occupe du titre de la section) tout en modifiant le contenu de chaque facture par la méthode `chargerfactureclients`.
- Et la dernière section qui liste les factures du client.

III. FLUX PRINCIPAL DE NOTRE APPLICATION

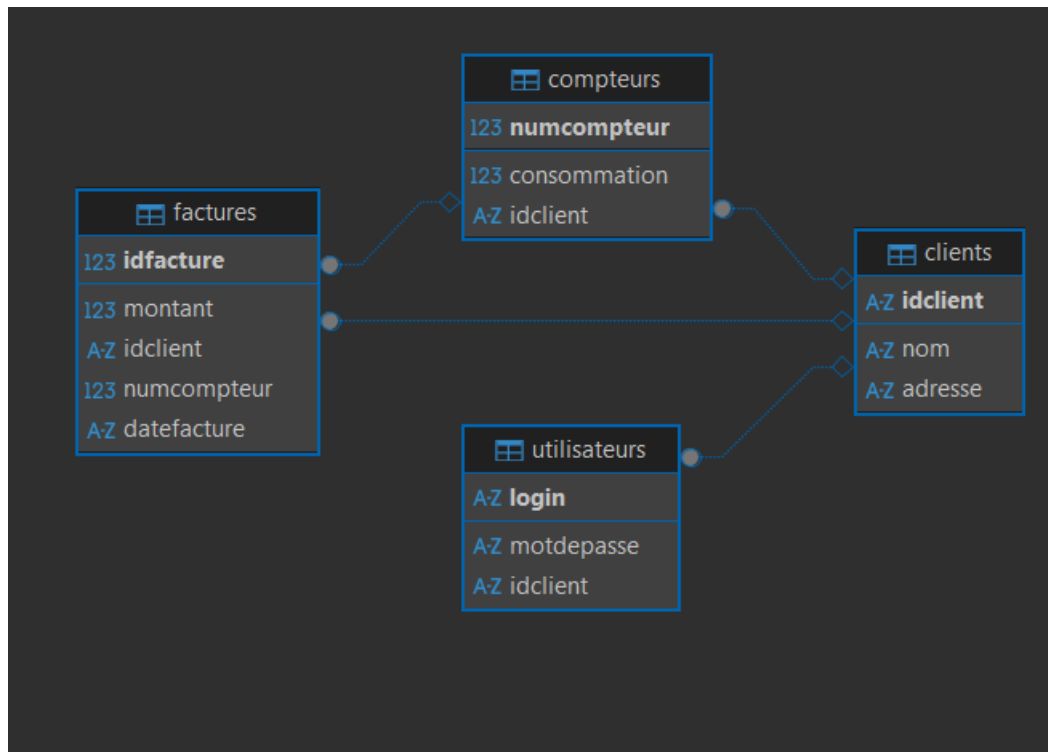
Le déroulement complet du flux principal lors de l'utilisation de l'application est comme si suit :

Au démarrage, on a la création de la BDD et sa gestion par les classes `InitDB` et `ConnexionDB`. Ensuite, l'ouverture de la fenêtre de connexion et inscription qui gère la vérification de l'ID et le mot de passe fait par la classe `ConnexionGU` qui aide aussi à la saisie des informations sur le client géré par la classe `Client` et à la génération de l'identifiant créée par la classe `Id`. Après ceci, l'interface de l'application EDI sera affichée par la classe `MainGU` alors le client devra saisir son numéro de compteur et sa consommation qui sont des données gérées par la classe `Compteur`. Cette interface va permettre le calcul du montant à payer grâce à la classe `Facture`. De même, la classe `MainGU` donne l'affichage du reçu de la facture sélectionnée, l'impression directe de la facture. Et la génération du PDF de la facture conduit par la classe `Facture`.

IV. BASE DE DONNEES

Voici la structure de notre base de données qui a été associée à EDI Energie

CREATION D'UNE APPLICATION JAVA : APPLICATION DE GESTION DE FACTURES D'ELECTRICITE : EDI ENERGIE



Le diagramme de notre base de données avec ces différentes tables

Nom de la table: clients Table Description: Table Type:

	Nom de la colonne	#	Type de donnée	Length	Non Null	Auto Increment	Défa
Colonne	123 idclient	1	INTEGER		[]	[v]	
Clefs	A-Z nom	2	TEXT		[]	[]	
Clefs étrangères	A-Z adresse	3	TEXT		[]	[]	

✚ La table Client : Cette table enregistre les données du client à savoir son nom et son adresse ainsi que son identifiant

CREATION D'UNE APPLICATION JAVA : APPLICATION DE GESTION DE FACTURES D'ELECTRICITE : EDI ENERGIE

Nom de la table:

compteurs

Table Description:

Table Ty

	Nom de la colonne	#	Type de donnée	Length	Non Null	Auto Increment
Colonnes Clefs Clefs étrangères	123 numcompteur	1	INTEGER		[]	[]
	123 consommation	2	REAL		[]	[]
	AZ idclient	3	TEXT		[]	[]

>

FK_compteurs_clients

compteurs

clients

FOREIGN KEY

CLIENTS PK

✚ La table Compteur : Qui utilise la clé primaire de la table Client en tant que clé étrangère et il enregistre le numéro de compteur et la consommation du client.

Nom de la table:

factures

Table Description:

Colonnes

Clefs

Clefs étrangères

Indexes

Références

Nom de la colonne	#	Type de donnée	Length	Non Null	Auto Increme
123 idfacture	1	INTEGER		[]	[v]
123 montant	2	REAL		[]	[]
123 idclient	3	INTEGER		[]	[]
123 numcompteur	4	INTEGER		[]	[]
AZ datefacture	5	TEXT		[]	[]

FK_factures_clients

factures

clients

FOREIGN KEY

CLIENTS_PK

No Action

FK_factures_compteurs_2

factures

compteurs

FOREIGN KEY

COMPTEURS_PK

No Action

✚ La table Facture : qui stocke le montant à payer, la date de la facture et l'identifiant de la facture pour le numéro de la facture. Il est en relation avec les 2 autres tables.

✚ La table Utilisateur : gère le compte

Nom de la table:

utilisateurs

Table Description:

Table T

Colonnes

Clefs

Clefs étrangères

Nom de la colonne	#	Type de donnée	Length	Non Null	Auto Increment
A-Z login	1	TEXT		[]	[]
A-Z motdepasse	2	TEXT		[]	[]
123 idclient	3	INTEGER		[]	[]

FK_utilisateurs_clients

utilisateurs

clients

FOREIGN KEY

CLIENTS_PK